Supplementary material: scope and reproduction of the partial Lean verification

Robert Hildebrand

October 21, 2026

This supplement accompanies *A short proof of a range-relative FPTAS for polynomial integer optimization over bounded polyhedra*. It explains which mathematical implications are checked in Lean and how to reproduce the checks. **The complete FPTAS, its implementation, and its running-time theorem are not formally verified.** The development contains 256 theorem declarations in eight modules; each retains its explicit hypotheses.

1. Paper-to-Lean map

Every named declaration has a complete Lean proof of its stated implication. Colors describe coverage of the paper, not unfinished proofs: **proved** marks the indicated construction or inequality; **conditional** marks a guarantee relying on imported mathematical or oracle hypotheses; **external** marks a claim not formalized here. Ordinary stated hypotheses apply throughout. Numbers refer to the short manuscript; representative declarations are in `verification/lean/`.

Paper argument	Module and principal declarations; checked conclusion
Lemma 2.1: reflection and integer samples **Proved**	**LineRestriction** `shifted_feasible_iff`, `shifted_slack_integral_progression`: the half-unit relaxation preserves integer feasibility, and narrow shifted-slack intervals supply feasible integer progression samples.
Lemma 2.3: coverage, cell descriptions, LP criterion **Geometry proved** **LP optimum assumed**	**CoverAssembly** `retained_slack_chamber_cover`, `chamber_closure_rational_description`, `chamber_closure_compressed`, `chamber_integer_tests`, `compressed_optimum_test`: retained closures cover the interior region, have the stated rational descriptions, and admit the rounded membership and conditional LP tests.
Lemma 2.3: chamber count **Count proved**	**ChamberInsertion** `insertion_inequality`, `arrangement_count_bound`, `shifted_chamber_count`: geometric insertion and dimension reduction yield the binomial count, including for parallel or repeated hyperplanes.
Lemma 2.5: sampled estimate **Even part proved** **Markov assumed**	**SampledMarkov** `even_contract`, `degree_contract`, `symmetrized_difference`, `sampled_markov`: construct the even part, bound its degree, and derive the discrete-minimum estimate from classical Markov hypotheses.
Proposition 2.6; Theorem 1.1 **Conditional guarantee** **Full algorithm external**	**LineRestriction; FPTASCalculations; CoverAssembly** `degree_restrict`, `multivariate_selection`; `ceiling_refinement`; `chamber_approximation`: actual polynomial restriction, the prescribed refinement, and cell coverage connect to the integer approximation guarantee, assuming complete feasible representatives.
Corollaries 1.2–1.3 **Scaling proved** **Conditional reduction**	**FPTASCalculations; CoverAssembly; ScaledPolynomial** `nonnegative_maximization`; `mixed_integer_reduction`; `integer_representation`, `feasible_bijection`: nonnegative maximization and a conditional mixed-integer reduction, including grid rounding, integer polynomial scaling and feasible-set identification.
Encoding and complexity calculations **Size bounds proved** **CHKM assumed** **Full runtime external**	**CoverArithmetic; ScaledPolynomial; CellEncoding** `chkm_encoding_translation`; `coefficient_bits`; `complete_lp_encoding`, `complete_lp_polynomial`, `rounded_cell_polynomial`: translate assumed geometric estimates and prove concrete encoding inequalities. The full algorithmic runtime is not certified.

Manuscript lemmas and their verification scope

Here are the three lemma statements from the short manuscript, followed by the representative proposition. Notation is that of the paper: $P = \{x : Ax \le b\}$ is bounded, $\widetilde{P} = \{x : Ax \le b + \frac{1}{2}\mathbf{1}\}$, $s_i(x) = b_i + \frac{1}{2} - a_i^T x$, and $\mathcal{C}$ consists of the closed refined shifted-slack chambers. The total input size is φ; τ controls row coefficient bit lengths and Φ is the paper's common coordinate/slack exponent. The statements below link to the corresponding Lean declarations at the pinned source revision. The following pages spell out their hypotheses.

Lemma 2.1 (Refined CHKM reflection property). Each $C \in \mathcal{C}$ is described by $a_i' \le s_i(x) \le c_i'$ for successive thresholds $a_i' < c_i'$. Set

$$D_C := \{v \in \mathbb{R}^d : |a_i^T v| \le a_i' \ (1 \le i \le m)\}.$$

Then $\ell(C - C) \subseteq D_C$ and $C \pm D_C \subseteq \widetilde{P}$. In particular,

$$x + k(y - x) \in P \cap \mathbb{Z}^d \quad (x, y \in C \cap \mathbb{Z}^d,\ k \in \mathbb{Z},\ |k| \le \ell).$$

Checked consequence: statement (A) below gives the integer progression explicitly. The actual chamber-width hypotheses are connected to approximation in (C); the displayed set inclusions are not asserted here as separately audited declarations.

LineRestriction.lean: `shifted_slack_integral_progression`; **CoverAssembly.lean**: `slack_chamber_narrow`.

Lemma 2.3 (Number, encoding, and enumeration of cells). The closed cells can be enumerated deterministically in $(\varphi\ell)^{O(d)}$ bit operations, and their number is at most $(\varphi\ell)^{O(d)}$. Each cell has a description by at most $2m$ non-strict linear inequalities of total encoding size $\mathrm{poly}(\varphi, \log(\ell + 1))$, hence $\mathrm{poly}(\varphi)$ for the prescribed refinement $\ell = \max\{D^2, \lceil D/\sqrt{\varepsilon} \rceil\}$. Rounding its slack bounds as specified in the paper gives a rational polytope $\widehat{C} \subseteq C$ with exactly the same integer points, described by at most $2m$ inequalities whose coefficient bit lengths are $O(\tau + \Phi + 1)$, independent of ℓ.

Checked components: the chamber count in (E), rational descriptions, compression, integer-membership tests and explicit encoding inequalities. **Not fully formalized:** deterministic enumeration and the total bit-operation bound. Thus the entire lemma is not labeled fully Lean verified.

ChamberInsertion.lean: `arrangement_count_bound`; **CoverAssembly.lean**: `chamber_closure_compressed`, `chamber_integer_tests`, `compressed_optimum_test`; **CellEncoding.lean**: `complete_lp_polynomial`, `rounded_cell_polynomial`.

Lemma 2.5 (Approximation at a discrete minimum). Let $D \ge 1$ and $\ell \ge D^2$ be integers, and let $p \in \mathbb{R}[t]$ have degree at most D. Suppose

$$a \le p(k) \le b \quad (k = -\ell, \dots, \ell), \qquad p(-1) \ge p(0).$$

Then $p(1) - p(0) \le D^2(b - a)/\ell^2$. **Verified assuming the two classical Markov inequalities:** statement (B) makes these external hypotheses explicit; the remaining argument is proved in Lean.

SampledMarkov.lean: `sampled_markov`.

Proposition 2.6 (Representatives approximate extrema). Choose one integer representative from each integer-feasible closed cell, forming Y. If $S := P \cap \mathbb{Z}^d \ne \varnothing$ and $\ell \ge D^2$, then every real polynomial f of degree at most D satisfies

$$0 \le \min_Y f - \min_S f \le \frac{D^2}{\ell^2}(f_{\max} - f_{\min}),$$

$$0 \le \max_S f - \max_Y f \le \frac{D^2}{\ell^2}(f_{\max} - f_{\min}).$$

Verified approximation ingredients: statement (C) gives the assembled minimum guarantee for the prescribed refinement, assuming complete feasible representatives and Markov. Applying the minimum argument to $-f$ gives the displayed maximum conclusion; this is the paper's mathematical specialization, not a claim of a separate two-sided declaration.

LineRestriction.lean: `shifted_slack_cover_guarantee`, `multivariate_selection`; **CoverAssembly.lean**: `chamber_approximation`.

2. Mathematical statements checked in Lean

The following are mathematical restatements of the indicated declarations, with finite coordinate sets written as Euclidean spaces. They distinguish assumptions from conclusions; they do not assert a verified executable algorithm. Write $S := \{x \in \mathbb{Z}^d : Ax \le b\}$, where A, b are integral, and $s_i(x) := b_i + \frac{1}{2} - a_i^T x$.

(A) Feasible integer progressions. Proved. Let $x, y \in S$ and $\ell > 0$. Suppose that, for each row i, there are real numbers a_i', c_i' such that

$$s_i(x), s_i(y) \in [a_i', c_i'], \qquad c_i' - a_i' \le a_i'/\ell.$$

Then

$$x + k(y - x) \in S \qquad \text{for every } k \in \mathbb{Z} \text{ with } |k| \le \ell.$$

This is `LineRestriction.shifted_slack_integral_progression`. No geometric oracle or Markov inequality is assumed in this statement.

(B) The sampled polynomial estimate. Verified assuming classical Markov bounds. Let $D, \ell \in \mathbb{N}$, $\ell \ge \max\{1, D^2\}$, and $p \in \mathbb{R}[t]$ have degree at most D. Assume

$$a \le p(k) \le b \quad (k \in \mathbb{Z},\ |k| \le \ell), \qquad p(0) \le p(-1).$$

The imported derivative inequalities are

$$\|q'\|_{[-\ell,\ell]} \le \frac{D^2}{\ell}\|q\|_{[-\ell,\ell]}, \qquad \|h'\|_{[0,\ell^2]} \le \frac{2\lfloor D/2 \rfloor^2}{\ell^2}\|h\|_{[0,\ell^2]},$$

for all polynomials q, h of degrees at most $D, \lfloor D/2 \rfloor$, respectively; $\|g\|_I := \max_{t \in I} |g(t)|$. Lean then proves

$$\boxed{p(1) - p(0) \le \frac{D^2}{\ell^2}(b - a).}$$

This is `SampledMarkov.sampled_markov`. The discrete-to-continuous comparison and symmetrization are proved, not additional assumptions.

(C) Approximation from the actual chamber cover. Verified assuming Markov and complete feasible representatives. Let $D \ge 1$, $\varepsilon > 0$, $\deg f \le D$, and $\ell := \max\{D^2, \lceil D/\sqrt{\varepsilon} \rceil\}$. Assume all rows of A are nonzero, $x^* \in S$, and

$$f(x^*) \le f(z) \le f_{\max}, \qquad s_i(z) < 2^\Phi \quad (z \in S,\ \text{all } i).$$

Let $\mathcal{C}$ be the shifted-slack chamber closures constructed in the paper with parameters Φ, ℓ. Suppose a finite set $Y \subseteq S$ satisfies

$$C \cap S \ne \varnothing \implies C \cap Y \ne \varnothing \qquad (C \in \mathcal{C}).$$

Under the Markov assumptions in (B), Lean proves that some $\widehat{x} \in \arg\min_{y \in Y} f(y)$ satisfies

$$\boxed{0 \le f(\widehat{x}) - f(x^*) \le \varepsilon\big(f_{\max} - f(x^*)\big).}$$

This is `CoverAssembly.chamber_approximation`. Coverage and the required relative widths of these actual chambers are proved within the development. The construction of Y by an implemented oracle is not verified.

2. Mathematical statements checked in Lean (continued)

(D) Mixed-integer error transfer. **Verified assuming slice data, Markov and a scaled solver guarantee.** Write $S_{\mathrm{mix}} := P \cap (\mathbb{R}^{d_c} \times \mathbb{Z}^{d_i})$. Let f have integer coefficients and degree at most $D \geq 1$, let $0 < \varepsilon \leq 1$, and let $x^* \in S_{\mathrm{mix}}$ satisfy $f(x^*) \leq f(z) \leq f_{\max}$ for all $z \in S_{\mathrm{mix}}$. Assume there are $r+1$ points $v_0, \ldots, v_r$ in a convex set $K \subseteq S_{\mathrm{mix}}$, with $r \geq 1$, sharing their integer coordinates, such that

$$x^* = \sum_{j=0}^{r} \lambda_j v_j, \quad \lambda_j \geq 0, \quad \sum_{j=0}^{r} \lambda_j = 1, \quad \lambda_0 \geq \frac{1}{r+1}, \quad (v_j)_c \in \Delta^{-1}\mathbb{Z}^{d_c}, \quad \Delta \in \mathbb{Z}_{>0}.$$

These are the supplied `SliceData`; their existence is not proved here. Set

$$k := \left\lceil \frac{4D^2 r(r+1)}{\varepsilon} \right\rceil, \quad Q := k\Delta, \quad S_Q := S_{\mathrm{mix}} \cap (Q^{-1}\mathbb{Z}^{d_c} \times \mathbb{Z}^{d_i}).$$

Assume classical Markov on $[0,1]$ for degree at most D. Lean constructs $y \in K \cap S_Q$ with

$$0 \leq f(y) - f(x^*) \leq \frac{\varepsilon}{2}\big(f_{\max} - f(x^*)\big).$$

Moreover, the scaled polynomial $g(u_c, u_i) := Q^D f(u_c/Q, u_i)$ has integer coefficients and degree at most D; scaling identifies its integer feasible set with S_Q. Suppose $u_{\min}, u_{\mathrm{ref}}, \widehat{u}$ are feasible scaled points, $u_{\min}$ minimizes g, and

$$g(\widehat{u}) - g(u_{\min}) \leq \frac{\varepsilon}{2}\big(g(u_{\mathrm{ref}}) - g(u_{\min})\big).$$

Then the decoded point $\widehat{x} = (\widehat{u}_c/Q, \widehat{u}_i)$ belongs to S_{mix} and

$$\boxed{0 \leq f(\widehat{x}) - f(x^*) \leq \varepsilon\big(f_{\max} - f(x^*)\big).}$$

The rounding and final transfer are `CoverAssembly.mixed_rounding_near` and `mixed_integer_reduction`; the polynomial representation and feasible-set identification are proved in `ScaledPolynomial`. For the paper's application, take u_{ref} to maximize g. The usual range-relative integer guarantee then supplies the displayed solver hypothesis. Such a maximizer exists because the nonempty scaled integer feasible set is finite; its value is a proof parameter, not an algorithmic input. For an arbitrary feasible reference point the hypothesis may be stronger: taking $u_{\mathrm{ref}} = u_{\min}$ would require an exact minimizer. The solver inequality is an input, not a certified implementation output.

(E) Geometric chamber count. **Proved.** For any convex set U in a real vector space of dimension d and any H affine hyperplanes, the number of nonempty strict sign chambers in U is at most

$$\sum_{j=0}^{d} \binom{H}{j}.$$

This is `ChamberInsertion.arrangement_count_bound`, with $\binom{H}{j} = 0$ for $j > H$. The geometric insertion inequality is proved; a program enumerating the chambers and its execution cost are not.

3. What the assumptions mean

Polynomial approximation. The core chain is

$$\text{relative slack widths} \implies \text{integer progressions} \implies \text{sampled error bound} \implies \text{representative error.}$$

The line-restriction and even-polynomial constructions are proved, rather than assumed. In particular, the formalized argument does not assume continuous stationarity at an integer minimizer.

`SampledMarkov.MarkovBound` supplies the two classical derivative inequalities used on $[-\ell, \ell]$ and $[0, \ell^2]$. They are explicit proposition-valued hypotheses, not added Lean axioms. The appropriately scaled classical derivative bounds are supplied as hypotheses. Degree contraction, nearest-integer comparison, the even-part identity, and downstream error algebra are checked; the affine rescaling of the classical Markov theorem is not derived here. The mixed-integer proof separately assumes the classical Markov inequality on $[0, 1]$.

What "complete feasible representatives" means. For $S = P \cap \mathbb{Z}^d$ and the actual chamber family $\mathcal{C}$, the input finite set Y must satisfy exactly two conditions:

$$\underbrace{Y \subseteq S}_{\text{every chosen point is feasible}} \quad , \quad \underbrace{\forall C \in \mathcal{C},\ C \cap S \neq \varnothing \Rightarrow C \cap Y \neq \varnothing}_{\text{every integer-feasible cell has a chosen point}} .$$

"Complete" refers to coverage of the cells, not to listing all integer points. A cell may have many feasible points; any one suffices. Extra feasible representatives are allowed, and a shared-boundary point may represent several cells. No representative is required for a cell with no feasible integer point. Neither local objective minimization nor knowledge of the global minimizer is required to choose these representatives.

The paper obtains such a set by enumerating all cells and calling integer feasibility on each one. Correct enumeration and correct oracle answers justify these two hypotheses; Lean does not run or certify those procedures. Once the hypotheses hold, the formal proof establishes the approximation guarantee. It does not assume that the chosen points are already good approximations.

Other hypotheses in the cover theorem. `CoverAssembly.chamber_approximation` also takes integral nonzero rows, continuity of their linear maps, a common upper slack bound, an attained integer minimizer and global objective bounds. Continuity is automatic for these finite-dimensional linear maps; extrema exist for a nonempty bounded integer feasible set. Zero-row preprocessing and the constant-objective branch are handled in the paper, not this theorem. The optimum and bounds are proof parameters: the algorithm need not be given their values.

Published geometric inputs. CHKM's coordinate and slack estimates are assumed. Their numerical translation into the manuscript's Φ is checked in `CoverArithmetic`. The assignment of the published per-row encoding convention to the manuscript input is a separate mathematical interface. Auxiliary determinant and denominator-clearing results verify additional algebra and size estimates, without certifying a complete encoded input parser.

Mixed-integer reduction. `CoverAssembly.SliceData` records convex-slice and compatible-denominator data, together with a Carathéodory representation with a largest barycentric weight. Their existence comes from the published integral-slice result and classical convex geometry. Given these data, the development checks upward rounding, the feasible segment, grid membership and the two error budgets. `mixed_integer_reduction` also assumes the scaled integer solver's approximation guarantee and the needed extrema/global bounds. The scaled integer polynomial and its feasible-set bijection are constructed in `ScaledPolynomial`. This is a conditional reduction, not a verified mixed-integer solver.

Encoding and costs. Signed integer size is modeled by one sign bit plus the binary magnitude length; rational size uses the reduced numerator and positive denominator. `CellEncoding` includes the compressed constraints, margin variable, margin bounds, objective and explicit metadata in a dense LP-query schema. It proves a conservative bound of $200(\varphi + 1)^5$ under explicit parameter-domination hypotheses. This verifies a size inequality, not a byte-level serializer or its execution cost. Bounds on operation counts and solver costs still require the paper's algorithmic analysis.

4. How to interpret the remaining assumptions

Established mathematics imported as hypotheses. The classical Markov inequalities supply derivative bounds for all polynomials in the indicated degree classes, not the approximation conclusion being proved. CHKM supplies coordinate/slack estimates, and the integral-slice and Carathéodory results supply the convex representation and common denominator. These are published mathematical inputs whose proofs are not reproduced in Lean. The downstream deductions from them are checked.

Algorithmic correctness supplied as a hypothesis. A finite representative set must satisfy the two conditions above. In the mixed-integer reduction, the scaled solver must return a feasible point satisfying the displayed $\varepsilon/2$ error bound. This is an interface to the integer approximation result, not an assumption of the desired mixed-integer conclusion. Similarly, the LP criterion assumes a feasible optimal LP solution; it proves how the optimum tests interior, but does not verify an LP solver.

Encoding hypotheses. The input parameters and coefficient bit lengths must be bounded by the chosen total input-size parameter. Lean checks explicit size inequalities from these bounds. A reader still checks that the paper's encoding convention supplies those bounds. No claim is made that a particular file parser, serializer or solver has been verified.

Remaining review obligations. A reader should still check the correspondence between the displayed Lean hypotheses and the manuscript; the cited CHKM and integral-slice inputs; exceptional cases handled separately in prose; the exact LP/ILP oracle guarantees; incremental chamber enumeration and its operation counts; and the full bit-complexity conclusions. The sharper parallel-family count and discussion of extensions are not supplied as complete formalized algorithms. Browser illustrations and finite-instance experiments provide examples, not general proofs or Lean-certified implementations.

A successful axiom audit checks for admitted proofs and unexpected axioms in the compiled declarations. It cannot show that their hypotheses hold in an application or that the formal statement exactly captures the intended manuscript claim. Lean's kernel, the toolchain and the surrounding software environment remain part of the trusted checking infrastructure.

5. Reproducing the verification

Repository. `https://github.com/RobertHildebrand/FPTAS-Polynomial-Optimization`

The checked Lean sources, checker scripts and pinned dependencies are available at this exact baseline commit:

`c056413fdc1ffb75934a57326803490fd548d92e`

The supplement maps those Lean sources to the revised short manuscript; subsequent editorial changes do not alter the checked Lean declarations. The run record hashes the listed manuscript sources, Lean sources and checking infrastructure. It does not hash this supplement or the submission PDFs; their source/PDF correspondence must be checked separately. Do not equate the baseline commit's older manuscript with the current submission PDF.

Pinned versions. Lean: `leanprover/lean4:v4.34.0-rc2`. Mathlib:

`85e3a25e006c35636f0e53b0e9296caca2685bc0`

Transitive package revisions are pinned in `verification/lean/lake-manifest.json`. Prerequisites are Git, Python 3 and `elan` on the executable search path. Installing the pinned dependencies requires network access; the audit itself does not install them.

```
git clone https://github.com/RobertHildebrand/FPTAS-Polynomial-Optimization.git
cd FPTAS-Polynomial-Optimization
git checkout c056413fdc1ffb75934a57326803490fd548d92e
cd verification/lean
lake exe cache get
cd ../..
python3 verification/check_all.py
```

Do not run `lake update`: it changes the dependency baseline. If an identical, already compiled package tree is available, the checker accepts `-packages-dir /absolute/path/to/.lake/packages`. A fresh checkout is preferable to changing dependency revisions to bypass a mismatch.

Expected result. All eight modules rebuild and pass the axiom audit, with 256 theorem declarations. The individual counts are:

FPTASCalculations	31	SampledMarkov	13
LineRestriction	15	ScaledPolynomial	64
CoverArithmetic	26	CoverAssembly	47
ChamberInsertion	21	CellEncoding	39

The checkers invoke `#print axioms` for every audited theorem and allow only `propext`, `Classical.choice` and `Quot.sound`. A compilation failure, missing report, `sorryAx` or any other axiom fails the audit. A plain `lake build` does not replace these checks. Historical files `ReflectingCells.lean` and `check_reflecting.py` are excluded from this suite.

Inspect the evidence. The aggregate runner writes `verification/last_run.json` with timestamps, file hashes, exit codes and success status. Individual logs are in `verification/lean/`. The command

```
python3 verification/check_all.py --check-record
```

compares the record with its explicitly listed current files, excluding this supplement and the submission PDFs; it does *not* rerun Lean. A stale record can reflect an editorial manuscript change, but must not be presented as certifying a new snapshot. Use the full audit to produce a new record after changes. Optional `-experiments` additionally runs exact finite-instance checks; these are separate from the formal verification.

Detailed theorem interfaces and all reproduction commands are maintained in the repository verification guide. The manuscript's AI-assistance disclosure also applies to the supporting formalizations and this explanatory supplement.